

An AI Driven Decision System for Value Aware Regression Testing

No Author Given

No Institute Given

Abstract. In agile software development, regression testing is an ongoing process activity. However, real-world time constraints necessitate selecting a subset of tests to run. Current regression test selection algorithms primarily focus on technical metrics such as requirement coverage while overlooking the business value each test validates. This study reframes regression test selection as a multi-objective optimization problem in which tests are selected to maximize business value within a constrained testing time while maintaining adequate requirement coverage. We apply an artificial intelligence-based search algorithm, and our results show that the proposed method consistently selects tests with higher business value than baseline approaches when time is limited, while maintaining comparable requirement coverage. These findings suggest that a driven value-aware selector can be incorporated into agile teams for decisions on allocating limited regression testing resources.

Keywords: Agile · Regression Testing · AI-Search Optimization.

1 Introduction

Agile now dominates software development, with over 70% of organizations adopting it for rapid, incremental value delivery [32]. Its success hinges on flexible planning, iterative feedback, and automated testing to maintain high quality. Continuous regression testing is essential to prevent new changes from breaking key functionality or stakeholder value [30,37,9,33], but the main challenge is selecting the right tests, not running them [43]. Most Regression Test Selection (RTS) research remains focused on technical metrics, often overlooking end-to-end business workflows and Business Value (BV), typically validated with User Acceptance Tests (UATs) [25,13]. Neglecting value-driven regression testing exposes agile teams to risks that automation alone cannot prevent, as seen in the 2024 CrowdStrike incident [8]. This work proposes a value-driven regression testing approach that guides UAT selection by business value, the core agile success metric [2].

In agile development, business value encompassing financial and non-financial outcome measures progresses and guides trade-offs [21]. BV links software to organizational goals, adapts to shifting priorities, and ensures each increment benefits stakeholders. Consistent with Lean Engineering [31], all activities, including regression testing, must preserve value. While regression testing does not create new value, it is essential for safeguarding delivered value in agile.

While agile’s emphasis on business value, most regression testing research relies on technical metrics such as test coverage, fault detection, or testing time, which assume equal business impact for all requirements [9]. While some studies weight requirements by priority [6,38], these weights rarely reflect actual post-delivery value. As Svensson et al. and Trujillo et al. show [39,41], high-priority features do not always equate to high value, and lower-priority ones can be highly beneficial. Thus, integrating business value into regression testing remains a key gap in agile.

Agile’s continuous delivery imposes strict time constraints, making regression testing inherently time-limited [30,37,9]. As systems grow, exhaustive testing after each change becomes infeasible [30,37,14]. Thus, regression testing becomes a resource allocation problem: teams must use limited testing time to maximize business value, not just technical metrics. This reframes regression testing as a multi-objective optimization selecting test subsets that maximize business value under time and coverage constraints [36,19]. Each subset is a trade-off among time, coverage, and value. Manual selection is inefficient and biased; agile’s shifting priorities require adaptive, evidence-based decision mechanisms.

This is where Artificial Intelligence (AI) based optimization becomes helpful. AI aids decision-making by selecting tests that maximize value and coverage within time limits. Search and learning-based algorithms evaluate test combinations for business value and time [18]. While ML can predict critical tests from historical data [30,37,43], such models often miss agile-specific factors like user stories and business value [30]. The focus is on optimizing test selection for cumulative business value within time constraints, supporting agile goals beyond defect detection.

Thus, this research addresses the lack of value-driven, time-efficient regression test selection in agile, aiming to align testing with stakeholder value. *The goal is to develop and empirically evaluate a value-driven, time-aware framework for regression test selection that operationalizes agile principles of business value preservation and time efficiency through intelligent, optimization-based decisions.* The paper is organized as follows: background, methods, validity, future work, and conclusion.

2 Background

This section covers relevant conceptual background that directly motivates this work and positions the gap in current research.

Agile regression testing safeguards validated functionality during rapid iteration and continuous integration [37]. Automated pipelines accelerate verification, but business value is rarely a core metric [9]. Recent advances include clustering for better fault detection [24], code coverage with change-impact analysis [1], dependency-based selection for faster feedback [26], and scalable continuous integration methods [14]. Class-level RTS is more precise but slower than module-level [35], and RTS is especially important for frequent, small code changes [43].

Regression Test Selection selects tests to ensure code changes do not break existing behavior, using static or dynamic analysis to identify those affected [25]. Safe approaches keep all tests that might reveal faults, omitting others for efficiency [33]. Classic RTS used dependency analysis (e.g., DejaVu, RETEST) [20], and has expanded to distributed version control, embedded, aspect-oriented, and GUI-based systems [17,4,12,5]. Increasingly, algorithms leverage requirements, linking them to validating tests [3,6,38]. Chittimalli et al. (2008) introduced requirement–test traceability and criticality-based prioritization; Srivastava et al. (2008) ranked tests using stakeholder-defined priority and risk, and Khalid et al. (2019) applied priority ranking to black-box test cases.

Search-Based Regression Testing (SBRT) applies Search-Based Software Engineering (SBSE) to automate test selection, prioritization, and minimization as search problems evaluated by fitness functions. Traditional SBRT often uses proxies like code coverage or execution cost, which may be unreliable [28]. Greedy algorithms are fast but get stuck in local optima; metaheuristics (Genetic Algorithms, Simulated Annealing, PSO) better explore the space. Li et al. (2007) found Genetic Algorithms consistently outperformed other heuristics [28]. De Souza et al. (2013) used Binary Constrained PSO for cost-aware test selection, greatly improving requirement coverage per unit cost over random and greedy baselines [36,16,23,18].

In summary, RTS has evolved from static and dependency-based to cost- and search-based methods, but most optimize only one criterion—coverage, fault detection, or cost—and rarely incorporate business value [25,28,36]. Agile teams must maximize business value, maintain coverage, and meet tight deadlines, yet existing RTS and SBRT rarely address all three. This creates risks of misaligned regression decisions and poor trade-offs between speed and critical functionality. This paper addresses the gap by formalizing regression testing as a value-aware, time-constrained multi-objective optimization and proposing a method that explicitly balances value, coverage, and time.

3 Methods

This section formalizes BV driven RTS as a multi-objective optimization balancing BV and requirement coverage within time constraints.

Problem Formulation and Approach: We formalize regression testing as follows: Figure 1a from Jorgenson [22] illustrates requirements (S), code (P), and tests (T). In any SDLC, a subset of requirements $S_i \subseteq S$ is implemented as P_i and tested by T_i . Regression testing targets only delivered, tested requirements (region 1 in Figure 1a). In each new version (P_{i+1}, T_{i+1}) , regression tests are those in $T_i \cap T_{i+1}$, ensuring shared requirements remain intact [33]. Due to time constraints [30,9], running all these tests is often impractical. RTS selects the most effective subset of $T_i \cap T_{i+1}$ within the Regression Test Window ($\Delta\tau$), prioritizing requirements present in both versions. Non-overlapping regions represent new or removed requirements and are excluded. The objective is to select tests that maximize business value (BV) and requirement coverage within RTW,

given each requirement's BV and execution time. Thus, time-aware BV-driven RTS is formulated as:

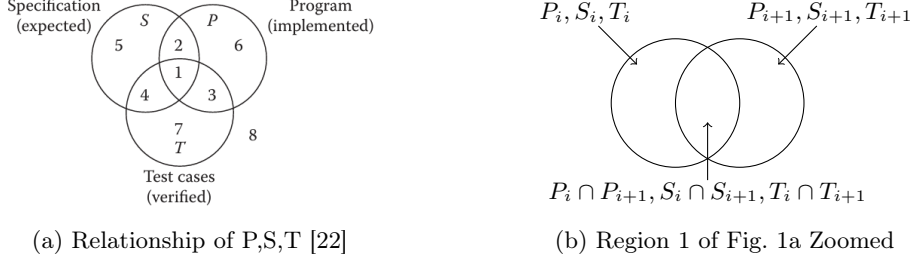


Fig. 1: Regression Test - Mathematical Definition

Problem Definition: Given the program versions P_i and P_{i+1} , the overlapping test pool $T_i \cap T_{i+1}$, the overlapping requirement set $S_i \cap S_{i+1}$, and the available regression test window $\Delta\tau$, the objective is to select subsets $(T_i \cap T_{i+1})' \subseteq (T_i \cap T_{i+1})$ and $(S_i \cap S_{i+1})' \subseteq (S_i \cap S_{i+1})$ that maximize the two criteria of (i) total BV $\sum_{s \in (S_i \cap S_{i+1})'} BV(s)$ and (ii) requirement coverage $\frac{|(S_i \cap S_{i+1})'|}{|(S_i \cap S_{i+1})|} \times 100$. The selection must satisfy the time constraint $\sum_{t \in (T_i \cap T_{i+1})'} ex(t) \leq \Delta\tau$, and each requirement s included in $(S_i \cap S_{i+1})'$ must have all of its validating tests selected; that is, $tests(s) \subseteq (T_i \cap T_{i+1})'$.

Algorithm: We implement our approach using Non-dominated Sorting Genetic Algorithm II (NSGA-II) [11,10], encoding solutions as binary vectors that indicate test selection. Fitness is assessed by two objectives: 1) total business value of covered requirements and 2) percentage of requirements in the regression test window covered. A population of feasible solutions is initialized within the execution-time budget $\Delta\tau$. Offspring are generated using tournament selection (k), crossover (p_c), and mutation (p_m), with a repair operator ensuring time constraints. NSGA-II then partitions solutions into Pareto fronts, using crowding distance to maintain diversity. This process is repeated for G generations, after which the first Pareto front provides trade-off solutions. Only requirements with all associated tests selected are counted, enforcing a strict ‘definition of done’ [29]. For evaluation, we select the solution with the highest business value, or, if tied, the one with the highest coverage, to compare with BCPSO and random algorithms. While the full Pareto front remains available for alternative goals, and teams can pick based on their quality goals, we report a single recommended test set per RTW for analysis.

Algorithm 1 NSGA-II for Test $\leftrightarrow$ Requirement Selection

```

1: Input: requirements' BV, tests' exec-time, budget  $\Delta\tau$ , population size  $N$ , generations  $G$ , tournament size  $k$ , crossover prob  $p_c$ , mutation prob  $p_m$ 
2: Sets:  $P$  population,  $O$  offspring,  $R$  combined pool
3:  $P \leftarrow \text{INITIALIZE-POPULATION}(N, s, \Delta\tau)$ 
4: for  $g = 1$  to  $G$  do
5:    $O \leftarrow \emptyset$ 
6:   while  $|O| < N$  do
7:      $p_1 \leftarrow \text{TOURNAMENT-SELECT}(P, k)$ 
8:      $p_2 \leftarrow \text{TOURNAMENT-SELECT}(P, k)$ 
9:      $(c_1, c_2) \leftarrow \text{CROSSOVER}(p_1, p_2, p_c)$ 
10:     $c_1 \leftarrow \text{MUTATE}(c_1, p_m)$ ;  $c_2 \leftarrow \text{MUTATE}(c_2, p_m)$ 
11:     $c_1 \leftarrow \text{REPAIR-FEASIBLE}(c_1, \Delta\tau)$ ;  $c_2 \leftarrow \text{REPAIR-FEASIBLE}(c_2, \Delta\tau)$ 
12:     $O \leftarrow O \cup \{c_1, c_2\}$ 
13:    $R \leftarrow P \cup O$ 
14:   Evaluate each  $r \in R$ :  $F_1 = \sum_s BV(s)$ ;  $F_2 = \frac{|\text{covered reqs}|}{|S_i \cap S_{i+1}|} \times 100$ 
15:    $F \leftarrow \text{NON-DOMINATED-SORT}(R)$ 
16:    $P \leftarrow \text{SELECT-NEXT-GENERATION}(F, N)$ 
17:  $PF \leftarrow$  first front of  $\text{NON-DOMINATED-SORT}(P)$ 
18: return  $PF$ 

```

4 Empirical Study

This section empirically evaluates the BV driven regression RTS. NSGA-II is compared to baseline and random selection methods for effectiveness in BV and requirement coverage. The research questions addressed are below:

RQ1: How does the proposed multi-objective algorithm balance BV and Requirement coverage under different RTWs?

RQ2: What is the BV/unit time of the proposed algorithm compared to BCPSO and Random?

Baseline: We evaluated our BV-driven RTS against a requirement-coverage baseline under similar time constraints. Most requirement-coverage RTS studies use test-centric metrics such as APFD or code coverage, but Kazmi et al. (2017) [25] found that requirement coverage is better for verifying satisfaction. De Souza et al. (2013) [36] provide a relevant baseline, using Binary Constrained PSO (BCPSO) to maximize requirement coverage within a time budget, treating coverage as the objective and cost as a constraint. For agile relevance, we require all tests for a requirement to pass for it to count as covered; partial coverage is insufficient, aligning with the definition of done. Our evaluation aggregates covered requirements and their cumulative BV as the metric.

Data: De Souza et al. (2013) [36] report aggregate execution-time budgets and measure solution quality by requirement coverage. Without a replication package, we reverse-engineered synthetic test sets, preserving the aggregate budget and mean pairwise Jaccard similarity (14.09%). For BV-driven RTS, 248 requirements were assigned Fibonacci business values (2–89) to reflect agile scaling [27,34], resulting in a right-skewed value distribution [40,7]. Test execution times were drawn from an exponential distribution with the observed mean, matching the baseline's total budget and realistic profiles [15]. To study input size, we

Parameter	Dataset 1	Dataset 2
# of Test Cases	80	40
Total Exec Time	699.31	349.65
Skewness (Exec Time)	3.26	2.84
Range (Exec Time, min-max)	1.16–101.14	1.27–79.41
# of Requirements	248	124
Total Business Value	3786	2480
Skewness (BV)	2.30	1.70
Range (BV)	2–89	2–89
Redundancy (%)	14.09	14.09

Table 1: Datasets

halved the number of test cases and requirements, holding other factors constant. This one-factor experiment isolates test set size as the independent variable [42]. *Note: Both datasets represent requirements and tests in $S_i \cap S_{i+1}, T_i \cap T_{i+1}$ of Figure 1b the unchanged behaviors across versions.*

Experiment Execution: Each algorithm was executed on the two datasets [Table 1] under varying RTW. Each configuration was repeated 30 times to account for the stochastic nature of the algorithms. The experimental setup is summarized below:

- Datasets: one re-created from the baseline and another created which has half number of requirements and tests, we call them 80-248 and 40-124 respectively.
- Algorithms: NSGA-II, BCPSO (baseline), and Random selection.
- RTWs: 5%-95% of total execution time in 5% increments (19 levels).
- Replications: 30 independent runs per configuration.
- Total executions: 3 algorithms $\times$ 19 levels $\times$ 30 replications = 1710 runs.
- NSGA-II Configuration: generations = 100, crossover probability = 0.9, mutation probability = $1/L$ where L is the chromosome length (i.e., the number of test cases)

Results: This section presents the results of our experiments. First plots and tables are presented followed by explanation of results.

Business value: In Figure 2a,b we see BV followed similar monotonic growth. NSGA-II consistently produced the highest BV throughout most RTWs and especially led between 10% and 40%. Baseline also increased steadily in BV, generally tracking just below NSGA-II, while Random produced substantially lower BV in early and mid RTWs. However, as RTW increased, Random’s BV improved and the performance gap narrowed. In the final RTW intervals, all three methods approached similar upper BV levels, with NSGA-II maintaining a small lead, Baseline just behind, and Random closing in but not exceeding the others.

Requirement coverage: Across both datasets in Figure 3a,b we see requirement coverage increased steadily for all methods as the RTW grew. NSGA-II and Baseline showed the fastest coverage gains early, both surpassing 50% by about 20% RTW, and tracked closely through the mid-range (approximately 30–60%

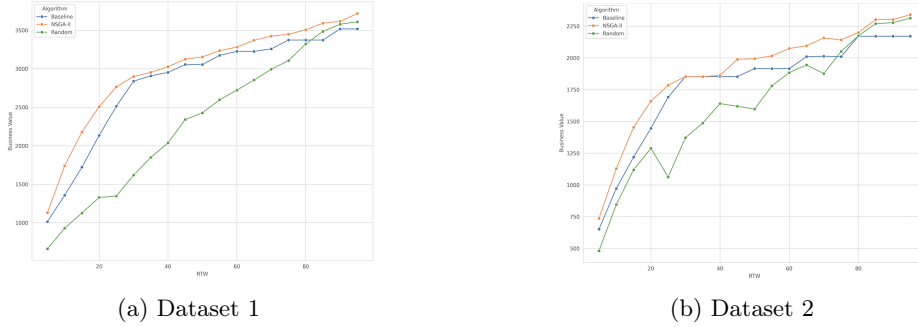


Fig. 2: Business Value achieved by NSGA-II, Baseline, and Random across RTWs

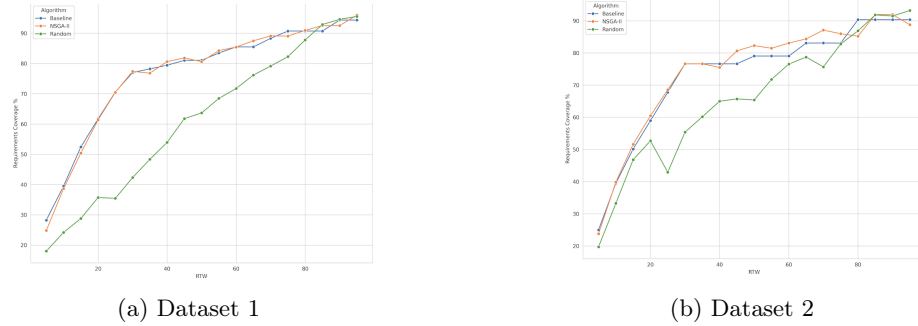


Fig. 3: Requirement Coverage achieved by NSGA-II, Baseline, and Random across RTWs

RTW), with NSGA-II holding slight advantages at several points. Random selection had slower early gains and lagged behind NSGA-II and Baseline through the first third of the RTW range, but its coverage improved steadily as RTW expanded. In the higher RTW intervals (above 75%), all methods converged, each exceeding 85–90% coverage, and differences became minimal.

Efficiency: Figure 4a,b shows that efficiency (BV per unit cost) steadily decreased as RTW increased across both datasets. At 10–15% RTW, Baseline and NSGA-II had the highest efficiency, with Baseline slightly behind; Random was much lower. The largest method separation occurred in this interval. From 20–65% RTW, all methods' efficiency declined, with Baseline and NSGA-II tracking closely and staying above Random, which dropped more sharply and plateaued well below the others. Above 70% RTW, efficiency values for all methods converged, and by 85–95% RTW, the three were nearly equal, with only minor differences across datasets.

One-way ANOVA revealed significant performance differences among NSGA-II, Baseline, and Random ($p < 0.001$), especially at low and mid RTWs. Tukey's HSD tests at 10%, 50%, and 90% RTW (shown due to space) indicate NSGA-

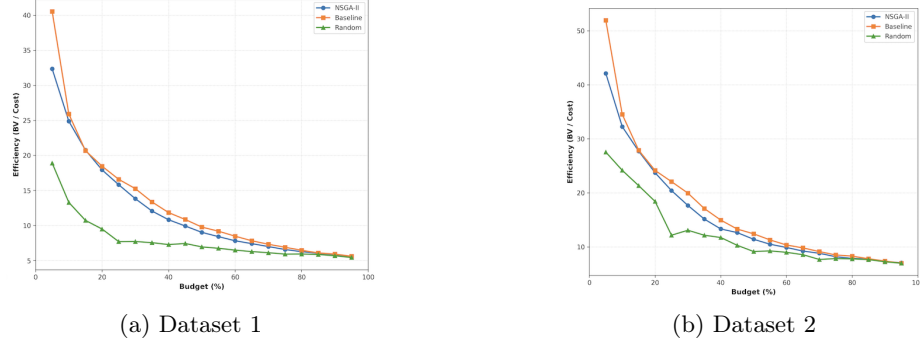


Fig. 4: BV/Unit Time by NSGA-II, Baseline, and Random across RTWs

Table 2: ANOVA results for Dataset 1 (D1) and Dataset 2 (D2).

RTW	Business Value		Requirement Coverage	
	D1	D2	D1	D2
5%	< 0.001	< 0.001	< 0.001	< 0.001
10%	< 0.001	< 0.001	< 0.001	< 0.001
15%	< 0.001	< 0.001	< 0.001	< 0.001
20%	< 0.001	< 0.001	< 0.001	0.004
25%	< 0.001	< 0.001	< 0.001	< 0.001
30%	< 0.001	< 0.001	< 0.001	< 0.001
35%	< 0.001	< 0.001	< 0.001	< 0.001
40%	< 0.001	< 0.001	< 0.001	< 0.001
45%	< 0.001	< 0.001	< 0.001	< 0.001
50%	< 0.001	< 0.001	< 0.001	< 0.001
55%	< 0.001	< 0.001	< 0.001	< 0.001
60%	< 0.001	< 0.001	< 0.001	0.009
65%	< 0.001	0.024	< 0.001	0.033
70%	< 0.001	< 0.001	< 0.001	< 0.001
75%	< 0.001	< 0.001	< 0.001	0.031
80%	< 0.001	0.541	0.012	< 0.001
85%	< 0.001	0.008	0.107	0.585
90%	< 0.001	< 0.001	< 0.001	0.465
95%	< 0.001	< 0.001	0.044	< 0.001

II and Baseline were closely matched on coverage, but NSGA-II consistently achieved higher business value. Full results are in the replication package

Random lagged behind in both metrics at lower and mid RTWs, narrowing the gap as budgets increased. Efficiency declined for all methods as RTW rose, with NSGA-II and Baseline most efficient at low RTWs and converging with Random at the highest budgets. These results confirm NSGA-II's advantage in balancing business value and coverage (see Tables 2 and 3).

5 Discussion

In this section, we present our interpretation of the results and answer the research questions.

RQ1. How does the proposed multi-objective algorithm balance business value and requirement coverage under different regression test windows (RTWs)?: The behavior of NSGA-II across RTWs suggests that regression testing in agile can

Table 3: Tukey HSD Comparisons for Business Value and Requirement Coverage for Dataset1(D1) and Dataset2(D2)

Business Value					
RTW	Pair	Mean Diff (D1)	95% CI (D1)	Mean Diff (D2)	95% CI (D2)
10%	Baseline vs Random	-427.33	[-564.74,-289.92]	-133.60	[-207.37,-59.83]
	NSGA-II vs Random	-810.10	[-947.51,-672.69]	-257.80	[-331.57,-184.03]
	Baseline vs NSGA-II	382.77	[245.36,520.18]	124.20	[50.43,197.97]
50%	Baseline vs Random	-688.50	[-852.30,-524.70]	-554.10	[-700.00,-208.20]
	NSGA-II vs Random	-689.70	[-854.40,-525.00]	-358.80	[-507.40,-210.20]
	Baseline vs NSGA-II	1.10	[-163.60,165.80]	-195.30	[-342.60,-48.00]
90%	Baseline vs Random	60.90	[21.80,100.00]	106.47	[36.78,176.16]
	NSGA-II vs Random	-36.83	[-75.94,2.27]	-24.53	[-94.22,45.16]
	Baseline vs NSGA-II	97.73	[58.63,136.84]	131.00	[61.31,200.69]
Requirement Coverage					
RTW	Pair	Mean Diff (D1)	95% CI (D1)	Mean Diff (D2)	95% CI (D2)
10%	Baseline vs Random	-15.3360	[-18.8848,-11.7872]	-6.2634	[-7.5147,-5.0121]
	NSGA-II vs Random	-14.4758	[-18.0246,-10.9270]	-6.5860	[-7.8373,-5.3347]
	Baseline vs NSGA-II	-0.8602	[-4.4090,2.6886]	0.3226	[-0.9287,1.5739]
50%	Baseline vs Random	-17.3790	[-21.9691,-12.7890]	-13.6559	[-19.8842,-7.4277]
	NSGA-II vs Random	-16.9624	[-21.5524,-12.3723]	-16.8817	[-23.1100,-10.6535]
	Baseline vs NSGA-II	-0.4167	[-5.0067,4.1734]	3.2258	[-3.0024,9.4541]
90%	Baseline vs Random	0.2285	[-0.9178,1.3748]	1.1559	[-2.0332,4.3451]
	NSGA-II vs Random	2.0565	[0.9102,3.2027]	-0.4570	[-3.6461,2.7322]
	Baseline vs NSGA-II	-1.8280	[-2.9742,-0.6817]	1.6129	[-1.5763,4.8021]

indeed be treated as a value-preserving allocation problem, rather than a purely technical coverage exercise. When the RTW is tight, the algorithm is forced to expose its “preferences”: it consistently converges on configurations that protect fewer, but more valuable requirements, while still respecting the requirement-level definition of done. This pattern indicates that the multi-objective formulation successfully internalizes Agile’s central principle of maximizing stakeholder value under constraints: instead of distributing time thinly across many low-impact behaviors, NSGA-II concentrates the available budget on high-value slices of functionality whose failure would be most damaging.

At the same time, the results show that this emphasis on value does not come at the expense of requirement coverage. Because the algorithm operates at the requirement level and counts a requirement as covered only when all of its validating tests are selected, the search is implicitly encouraged to form cohesive “bundles” of tests that complete requirements, rather than scattered partial selections. This creates a natural coupling between value and coverage, i.e., high-value requirements are protected end-to-end, while the algorithm continues to explore solutions that expand coverage as more time becomes available. The fact that NSGA-II tracks the requirement coverage driven baseline closely, while often achieving higher cumulative business value, indicates that value and coverage can be jointly optimized rather than treated as competing objectives.

The way NSGA-II behaves across the RTW range also reveals something about different operating regimes for agile teams. In very small RTWs, the algorithm’s selections embody a “core value shield”. The pipeline has just enough time to guard the most valuable behaviors, while accepting that some peripheral functionality will remain temporarily untested. As the RTW grows into a mid-range, NSGA-II’s solutions become more balanced, progressively incorporating medium-value requirements and thereby expanding coverage without losing sight of value. Once the RTW is large enough to admit most tests, the optimization problem becomes almost degenerate, and all approaches converge to similar BV and coverage, with the specific balancing strategy mattering far less. This reinforces that intelligent selection is most needed exactly where agile teams feel the most pressure when feedback must be fast, and budgets are tight.

From a conceptual standpoint, the observed balance between BV and requirement coverage suggests that RTS should be explicitly framed as a multi-objective decision rather than a single-metric optimization. NSGA-II’s search does not commit to a single “best” trade-off; it generates a frontier of alternatives that differ in how they balance value concentration versus breadth of coverage. In practice, this gives product owners and QA leads a spectrum of defensible choices, for example, preferring a solution that best suites their quality goals. In this way, the algorithm does not replace human judgment; it structures it, making explicit the trade-offs that are often handled informally or heuristically in current agile regression practice. Moreover, this philosophy also aligns with the concept of using value as metric for regression testing along with the current technical metrics we have as mentioned by Das and Gary 2025 [9].

Finally, the fact that the same qualitative balancing behavior emerges on two datasets of different sizes indicates that the approach is not overly sensitive to the scale of the regression test set. Whether the team faces a smaller or larger pool of UATs, NSGA-II consistently allocates limited test time to protect high-value requirements while maintaining competitive coverage. This supports the broader claim that value-aware, time-constrained multi-objective optimization is a viable foundation for RTS in agile, rather than merely an artifact of a particular dataset or configuration.

RQ2. What is the BV/unit time of the proposed algorithm compared to BCPSO and Random?: Interpreting *BV per unit time as efficiency = (total BV of requirements whose all tests are selected / execution time of selected tests)*. The results show that NSGA-II and the BCPSO baseline are broadly similar in efficiency, and both clearly outperform Random, especially at low and mid RTWs. As RTW grows, all three methods exhibit steadily declining BV/unit time and eventually converge, reflecting the expected diminishing returns: once most requirements are covered, each extra minute of testing buys proportionally less additional value. From an agile perspective, this reinforces the idea that early minutes are much more valuable than late ones.

The more interesting behavior appears in the small and mid RTWs, where we sometimes observe that the baseline achieves slightly higher BV/unit time than NSGA-II, even though NSGA-II delivers more total BV over the same RTW grid.

This is not a contradiction; it is a direct consequence of what each algorithm is optimizing. NSGA-II is explicitly tuned to maximize (i) total business value and (ii) requirement coverage, under the hard time constraint, with a strict “definition of done” that only counts a requirement if all its tests are executed. In practice, this means NSGA-II is willing to “pay” for some high-value but relatively expensive requirements if they help the solution move up on the BV axis, even when their test bundles consume a substantial portion of the budget. BCPSO, in contrast, is designed to maximize requirement coverage within a given cost; it naturally favors cheaper requirements and test clusters, because they let it cover more distinct requirements within the same time budget. When we then compute BV/unit time using the actual execution time of the selected tests, this coverage-centric behavior can yield slightly higher value density per second for BCPSO, even though the absolute value protected is lower.

The characteristics of the datasets accentuate this effect. Business value assignments follow a right-skewed distribution, and execution times are sampled from a right-tailed exponential distribution, with no enforced correlation between the two, following the baseline’s idea. As a result, there exist clusters of cheap, medium-BV requirements and expensive, high-BV requirements. NSGA-II’s BV objective pulls it toward including some of the expensive, high-BV requirements once the “cheap head” is mostly exploited, whereas BCPSO continues to accumulate many low-to-medium value requirements with short tests. In pure efficiency terms, the baseline is effectively skimming “cheap value” first, so its BV/unit-time curve can sit slightly above NSGA-II’s in some bands, even as NSGA-II secures a larger share of the total BV available in the system.

Seen through an agile lens, this trade-off is meaningful rather than problematic. A team that optimized solely for BV per minute could end up with a portfolio dominated by small, inexpensive stories that collectively look very efficient but leave some high-stakes workflows under-tested. NSGA-II, on the other hand, behaves more like a product owner under time pressure. It uses the available window to ensure that a larger portion of the release’s high-value requirements are fully exercised, accepting that the resulting test set may not maximize BV/unit time at every RTW. The efficiency results, therefore, suggest that our multi-objective approach is making a deliberate trade: it sacrifices a small amount of value density per second to protect more of what matters most, while remaining within the same efficiency range as a cost-sharpened coverage baseline and well ahead of uninformed selection. For agile teams, that trade-off aligns with the core goal of regression testing: not to win an efficiency metric in isolation, but to spend limited regression time in a way that most robustly preserves delivered business value.

6 Threats to Validity

In this section, we discuss potential threats to the validity of our study along the four common dimensions of internal, construct, external, and conclusion validity.

Internal validity may be affected by using synthetic datasets and assumptions about BV and execution-time distributions, as well as dataset scaling and NSGA-II parameter choices, although replication and literature-backed settings help mitigate some of this risk.

Construct validity is limited by the strict requirement-level “definition of done” and the Fibonacci-based BV distribution, which may differ from real-world practices.

External validity is constrained by the use of synthetic datasets and a narrow set of baselines, which may reduce generalizability to industrial UAT suites and diverse agile teams.

Conclusion validity is supported by statistical significance testing (ANOVA, Tukey HSD), but is weakened by the absence of effect sizes and adjustments for multiple comparisons; replication and trend consistency support our conclusions, though additional statistical detail would further increase confidence.

Overall, these threats indicate that our results should be interpreted as an initial, controlled evaluation of value-aware, time-constrained regression test selection rather than definitive evidence of performance in all industrial settings. Nonetheless, the consistency of trends across replications and the use of literature-informed parameters suggest that the observed advantages of the proposed approach are meaningful and warrant further investigation with real-world datasets and broader baselines.

7 Conclusion and Future Work

In conclusion, this work re-frames agile regression testing as a business value preservation problem, formalizes it as a multi-objective optimization maximizing BV and requirement coverage within a finite RTW, and demonstrates it with an NSGA-II-based approach. Empirical results show that, especially under tight and mid-range budgets, the method consistently protects more business value while maintaining competitive coverage, converging with other methods only when time is abundant. The core contribution is conceptual: for the first time, AI and business metrics are combined for test and resource management, positioning regression selection as an AI-supported, value-focused decision. AI makes trade-offs explicit, supporting agile’s goal of continuously safeguarding delivered stakeholder value.

Moreover, future work will enhance AI-based decision-making and data quality. NSGA-II can evolve into value-aware optimization services integrated into CI/CD pipelines, continuously providing pareto-optimal test sets as decision-support tools. Here, the RTS engine acts as a service, recommending test subsets per change based on business value, coverage, and time, and making trade-offs transparent for stakeholders. A key challenge is the “starvation effect”: optimizing cycles in isolation can cause some requirements to be repeatedly skipped. Addressing this requires richer data—tracking cross-cycle history and recency, adding anti-starvation constraints, and collaborating with industry for longitudinal agile datasets with requirements, tests, BV, and pipeline telemetry. Ulti-

mately, AI needs only business value as a semantic signal, quantified however an organization chooses, as measurement itself shapes what is observed [21].

Acknowledgment: This work acknowledges the assistance of a Large Language Model (LLM) used explicitly for grammar and spelling-checking purposes. The author has reviewed all content to ensure accuracy, integrity, and correctness.

References

1. Ali, S., Hafeez, Y., Hussain, S., Yang, S.: Enhanced regression testing technique for agile software development and continuous integration strategies. *Software Quality Journal* **28**, 397–423 (2020). <https://doi.org/10.1007/s11219-019-09492-2>
2. Alliance, A.: Manifesto for agile software development (2001), <https://agilemanifesto.org>, retrieved September 12, 2024
3. Arafeen, M.J., Do, H.: Test case prioritization using requirements-based clustering. In: *Proceedings of the 2013 IEEE 6th International Conference on Software Testing, Verification and Validation*. pp. 312–321. IEEE (March 2013)
4. Biswas, S., Mall, R., Satpathy, M.: A regression test selection technique for embedded software. *ACM Transactions on Embedded Computing Systems* **13**, 47:1–47:39 (2013). <https://doi.org/10.1145/2539036.2539043>, https://consensus.app/papers/a-regression-test-selection-technique-for-embedded-biswas-mall/bc0778b835c25d9ebf6a741de1978a15/?utm_source=chatgpt
5. Chesser, C.: A Regression Test Selection Technique for Graphical User Interfaces. Ph.D. thesis, Kansas State University (2012), https://consensus.app/papers/a-regression-test-selection-technique-for-graphical-user-chesser/9b84cefd3cb2510eb62acba64c1cddf2/?utm_source=chatgpt
6. Chittimalli, P.K., Harrold, M.J.: Regression test selection on system requirements. In: *Proceedings of the 1st India Software Engineering Conference*. pp. 87–96. ACM (February 2008)
7. Cottmeyer, M.: Small agile user stories reduce variability and improve predictability (January 2014), <https://www.leadingagile.com/2014/01/small-stories-reduce-variability-velocity-improve-predictability>, accessed: 2025-10-23
8. CrowdStrike: Channel file 291 incident root cause analysis. Tech. rep., CrowdStrike, Inc., Austin, TX (2024), <https://www.crowdstrike.com/wp-content/uploads/2024/08/Channel-File-291-Incident-Root-Cause-Analysis-08.06.2024.pdf>, retrieved from CrowdStrike website
9. Das, S., Gary, K.: Regression testing in agile—a systematic mapping study. *Software* **4**(2), 9 (2025)
10. Deb, K.: An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering* **186**(2–4), 311–338 (2000)
11. Deb, K.: Multi-objective optimisation using evolutionary algorithms: an introduction. In: *Multi-objective Evolutionary Optimisation for Product Design and Manufacturing*, pp. 3–34. Springer London, London (2011)
12. Delamare, R., Baudry, B., Le Traon, Y.: Regression test selection when evolving software with aspects. In: *ACM SIGSOFT International Symposium on Software Testing and Analysis*. p. 7 (2008). <https://doi.org/10.1145/1404953.1404960>, <https://consensus.app/papers/>

- regression-test-selection-when-evolving-software-with-dela-mare-baudry/67d5b2447c425a51b2ed22ecc9456bbf/?utm_source=chatgpt
13. Edin, A., Qorbanzada, M.: An ambidextrous approach to user acceptance testing management and automation: How software customers can optimize their processes to explore new technology as well as current performance to improve the effectiveness of user acceptance testing (2022)
 14. Elbaum, S., Rothermel, G., Penix, J.: Techniques for improving regression testing in continuous integration development environments. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. pp. 235–245. ACM, New York, NY, USA (November 2014)
 15. Elbaum, S., Malishevsky, A.G., Rothermel, G.: Test case prioritization: A family of empirical studies. *IEEE Transactions on Software Engineering* **28**(2), 159–182 (2004)
 16. Elbeltagi, E., Hegazy, T., Grierson, D.: Comparison among five evolutionary-based optimization algorithms. *Advanced Engineering Informatics* **19**(1), 43–53 (2005)
 17. Gligorić, M., Majumdar, R., Sharma, R., Eloussi, L., Marinov, D.: Regression test selection for distributed software histories. In: International Conference on Fundamental Approaches to Software Engineering. pp. 293–309 (2014). https://doi.org/10.1007/978-3-319-08867-9_19, https://consensus.app/papers/regression-test-selection-for-distributed-software-gligorić-majumdar-f5cf99835d7d5a42b456a21daa8f4304/?utm_source=chatgpt
 18. Harman, M., Jones, B.F.: Search-based software engineering. *Information and Software Technology* **43**(14), 833–839 (2001)
 19. Harman, M., McMinn, P.: A theoretical and empirical study of search-based testing: Local, global, and hybrid search. *IEEE Transactions on Software Engineering* **36**(2), 226–247 (2009)
 20. Harrold, M.J., Jones, J.A., Li, T., Liang, D., Orso, A., Pennings, M., Sinha, S., Spoon, S., Gujarathi, A.: Regression test selection for java software. In: Proceedings of the 16th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications. pp. 312–326 (2001). <https://doi.org/10.1145/504282.504305>, https://consensus.app/papers/regression-test-selection-for-java-software-harrold-jones-71258751df6e54df983339c8c182f60e/?utm_source=chatgpt
 21. Heidenberg, J., Weijola, M., Mikkonen, K., Porres, I.: A model for business value in large-scale agile and lean software development. In: Systems, Software and Services Process Improvement: 19th European Conference, EuroSPI 2012, Vienna, Austria, June 25–27, 2012. Proceedings. vol. 19, pp. 49–60. Springer Berlin Heidelberg, Berlin, Germany (2012)
 22. Jorgensen, P.C.: *Software Testing: A Craftsman’s Approach*. Auerbach Publications, Boca Raton, FL, USA (2013)
 23. Kachitvichyanukul, V.: Comparison of three evolutionary algorithms: Ga, pso, and de. *Industrial Engineering and Management Systems* **11**(3), 215–223 (2012)
 24. Kandil, P., Moussa, S., Badr, N.: Cluster-based test cases prioritization and selection technique for agile regression testing. *Journal of Software: Evolution and Process* **29**(6), e1794 (2017). <https://doi.org/10.1002/smr.1794>
 25. Kazmi, R., Jawawi, D.N., Mohamad, R., Ghani, I.: Effective regression test case selection: A systematic literature review. *ACM Computing Surveys (CSUR)* **50**(2), 1–32 (2017)
 26. Koivuniemi, J.: Shortening feedback time in continuous integration environment in large-scale embedded software development with test selection. Master’s thesis, Master’s Thesis, J. Koivuniemi (2017)

27. Leffingwell, D.: Agile Software Requirements: Lean Requirements Practices for Teams, Programs, and the Enterprise. Addison-Wesley Professional (2010)
28. Li, Z., Harman, M., Hierons, R.M.: Search algorithms for regression test case prioritization. *IEEE Transactions on Software Engineering* **33**(4), 225–237 (2007)
29. Nidhra, S., Dondeti, J.: Black box and white box testing techniques: A literature review. *International Journal of Embedded Systems and Applications (IJESA)* **2**(2), 29–50 (2012)
30. Pan, R., Bagherzadeh, M., Ghaleb, T.A., Briand, L.: Test case selection and prioritization using machine learning: a systematic literature review. *Empirical Software Engineering* **27**(2), 29 (2022). <https://doi.org/10.1007/s10664-021-10046-1>
31. Poppendieck, M.: Principles of lean thinking. *IT Management Select* **18**, 1–7 (2011)
32. of Project Management, I.: 16th annual state of agile report (2022), <https://instituteofprojectmanagement.com/wp-content/uploads/2023/01/AR-SA-2022-16th-Annual-State-Of-Agile-Report.pdf>, retrieved September 12, 2024
33. Rothmel, G., Harrold, M.J.: A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology (TOSEM)* **6**(2), 173–210 (1997)
34. Salem, A.B.: What is wsjf – weighted shortest job first safe? (January 2022), https://scrum-master.org/en/what-is-wsjf-weighted-shortest-job-first-safe/?utm_source=chatgpt.com, accessed: 2025-10-23
35. Shi, A., Zhao, P., Marinov, D.: Understanding and improving regression test selection in continuous integration. In: *Proceedings of the 2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. pp. 228–238. IEEE, New York, NY, USA (October 2019)
36. de Souza, L.S., et al.: Search based constrained test case selection using execution effort. *Expert Systems with Applications* **40**(12), 4887–4896 (2013)
37. Spieker, H., Gotlieb, A., Marijan, D., Mossige, M.: Reinforcement learning for automatic test case prioritization and selection in continuous integration. In: *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. pp. 12–22. ACM, New York, NY (2017)
38. Srivastva, P.R., Kumar, K., Raghurama, G.: Test case prioritization based on requirements and risk factors. *ACM SIGSOFT Software Engineering Notes* **33**(4), 1–5 (2008)
39. Svensson, R.B., Torkar, R.: Not all requirements prioritization criteria are equal at all times: A quantitative analysis. *Journal of Systems and Software* **209**, 111909 (2024)
40. Team, I.A.J.E.: Estimating the business value of stories (2020), <https://integralagile.com/journal/estimating-business-value-stories>, accessed: 2025-10-23
41. Trujillo, T.: Business value and prioritization: What work should you do first? (2023), <https://www.catalyte.io/insights/business-value-and-prioritization-what-work-should-be-done-first/>, accessed: 2025-08-07
42. Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A.: *Experimentation in Software Engineering*, vol. 236. Springer, Berlin (2012)
43. Yu, T., Wang, T.: A study of regression test selection in continuous integration environments. In: *Proceedings of the 2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*. pp. 135–143. IEEE (October 2018)